RAG: Baseline Retrieval vs. Cross-Encoder Re-Ranking

Shreyas Deb | Roll No. 2023113024 https://github.com/shrey715/shiori

Setup

Document The ICC T20 World Cup 2026 playing conditions. I picked them because they are full of "unless / except / shall not" clauses, and the appendices (DRS, Super Over, warm-up matches, reserve day) override the main rules. The PDF is from February 2026, so the LLM can't have seen it.

Chunking Split by clause number (e.g. 9.1, 9.1.1), consecutive clauses of a section merged up to ∼180 tokens; headers, diagram labels and blank forms dropped. **440 chunks**, kept small on purpose: with big chunks the answer is almost always in the top 3 already. Every chunk exists as **A** (plain text) and **B** (section title + text).

Models Bi-encoders: `all-MiniLM-L6-v2`, `bge-small-en-v1.5` (FAISS, exact cosine). Re-rankers: `ms-marco-MiniLM-L6-v2` (used for Answer 2), `bge-reranker-base` (rankings only). LLM: `Qwen3-32B-AWQ` on vLLM over 4×RTX 2080 Ti, temperature 0.

Questions 40 questions in 4 tiers of 10, each harder than the last: direct wording, fan paraphrase, exceptions/negations, traps/cross-references. Each has a gold chunk and an expected answer, checked against the rule text.

Grading Every answer (No RAG, Answer 1, Answer 2) graded against the expected answer as correct (1), partial (0.5), wrong or abstain (0), by the same LLM with a fixed rubric. A blind second grader (Claude Haiku) agreed exactly on 79% and within one step on 97%; I fixed the 5 clear grader errors by hand.

1 Bi-encoder vs. Cross-encoder

Both of my models use the *same* backbone: a 6-layer MiniLM transformer (384 dims, 12 attention heads). The difference is only in **how the question and the chunk are fed in** (Figure 1), and that changes everything about cost and accuracy (Table 1).

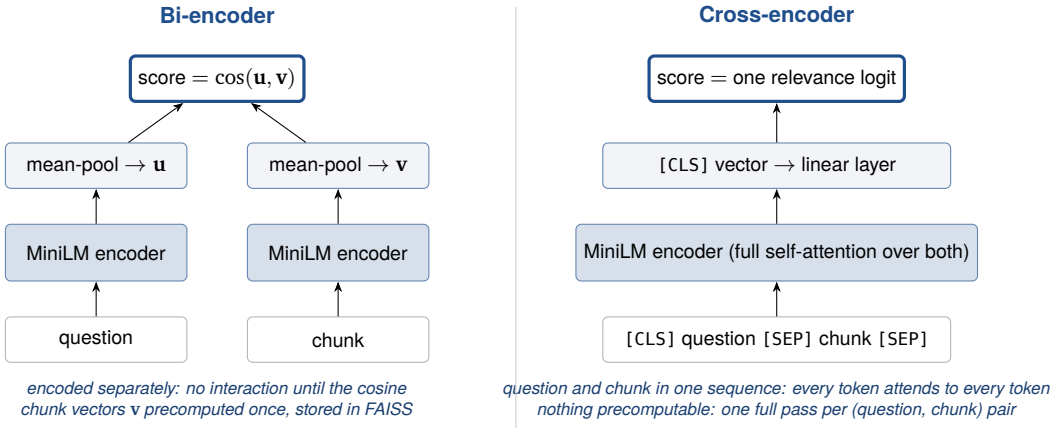


Figure 1: Same transformer, two ways of using it.

$$\text{bi-encoder: } s(q,d) = \cos\big(E(q),\, E(d)\big), \quad E(x) = \text{mean-pool}\big(\text{MiniLM}(x)\big)$$
$$\text{cross-encoder: } s(q,d) = \mathbf{w}^\top \mathbf{h}_{\texttt{[CLS]}}\big(\text{MiniLM}(\texttt{[CLS]}\ q\ \texttt{[SEP]}\ d\ \texttt{[SEP]})\big)$$

Why it matters, with a real example (q35). For "In a 7-overs-a-side rain game, what's the most a single bowler can send down?" the bi-encoder ranked clause 13.9.1 ("No bowler shall bowl more than 4 overs") first, cosine 0.575, and the actual answer, 13.9.2.2 ("between 5 and 9 overs … no more than two overs"), fourth, cosine 0.519. Both chunks are about overs per bowler, so their vectors sit close to the question; the single vector has no room for the "5 to 9 overs" condition. The cross-encoder reads "7-overs" next to "between 5 and 9 overs" in the same attention pass and flips them: 13.9.2.2 scores -3.34 (#1) vs. -4.74 for 13.9.1. Same model size, but because the two texts meet inside the network, it can check the condition instead of just the topic.

aspect	Bi-encoder	Cross-encoder
input	question and chunk encoded *separately*	question and chunk joined into *one* sequence
interaction	none until the final dot product ("late")	in every layer, each question token attends to each chunk token ("early")
output	two 384-d vectors, compared by cosine	one relevance score
precompute	all chunk vectors, once	nothing, the score depends on the question
cost per query	1 encoder pass + N dot products	N encoder passes
trained to	put sentences with similar meaning close together	judge whether this passage answers this query (MS MARCO labels)
good at	fast recall of the right *topic*	fine detail: conditions, exceptions, negation

Table 1: What the architecture difference means in practice.

2 Qualitative & Rank Shift Analysis

2.1 Overall

Re-ranking improved MRR in all 4 setups and hit@3 in 3 of 4 (Table 2). Over all setups it moved the gold chunk from rank 4–10 into the top 3 in **16 cases**, and pushed it out of the top 3 in 9. 15 of the 16 fixes are in tiers 3–4; tier 1 was already 100%.

setup	**hit@3** base → re-ranked	**MRR@10** base → re-ranked	**answer score** A1 → A2
MiniLM, plain (A)	0.65 → **0.70**	0.53 → **0.65**	**0.68** → 0.65
MiniLM, titled (B)	0.62 → **0.68**	0.50 → **0.65**	0.59 → **0.64**
bge, plain (A)	0.70 → **0.78**	0.63 → **0.69**	0.65 → **0.72**
bge, titled (B)	0.75 → 0.75	0.60 → **0.69**	0.70 → **0.72**

Table 2: Retrieval and answer quality before and after re-ranking with `ms-marco` (40 questions).

The bigger `bge-reranker-base` (278M params) was about the same as the 22M `ms-marco` one, so a bigger re-ranker didn't buy much here. Adding section titles to chunks (B) didn't help consistently either.

2.2 Failure cases fixed by re-ranking

In every case below the important chunk sat outside the top 3, so the LLM in Pipeline 1 never saw it.

question	type	gold chunk	Answer 1 (baseline)	Answer 2 (re-ranked)
q23: can a concussed player be replaced by someone not on the list?	exception	c008 (1.2.7.1.3.4) **#9 → #3**	"No, a concussed player can only be replaced by a player who is on the nominated list." **Omits the exception.**	"Yes, … but only under exceptional circumstances. … the ICC Match Referee will consider the request." Correct.
q35: 7-over rain game, max overs per bowler?	buried chunk	c091 (13.9.2.2) **#4 → #1**	"1 over … no bowler may bowl more than one-fifth" (the rule for 10+ overs). **Misstates the rule.**	"2 overs, as per rule 13.9.2.2." Correct.
q28: can the non-striker sneak a run during the run-up?	negation	c314 (41.16) **#5 → #2**	"Not in the context." (top 3 were the wrong run-out clauses)	"No … Dead ball, … disallow the attempted run, and award 5 Penalty runs." Correct.
q36: start time of the final on the reserve day?	keyword trap	c069 (12.7.1) **#4 → #2**	"The final will start … when the coin toss takes place", no time given. **Made up.**	"The final starts at 19:00." Correct.
q38: free hit called wide, can he be stumped?	exception over 2 chunks	c175 #4 → #3, c174 **#6 → #2**	"Yes, … the batter can be out stumped." **Wrong:** uses the general wide rule.	"No, … only … the circumstances that apply for a No ball, even if … called Wide." Correct.

Table 3: Showcase failures (setups: q23, q35 MiniLM-A; q28 MiniLM-B; q36, q38 bge-A).

So the baseline failed in exactly the ways the assignment describes: it **left out an exception** (q23, q38), **applied the wrong rule** (q35), **made something up** (q36), or gave up because the right clause wasn't there (q28). In 12 of the 16 rank fixes the answer improved too.

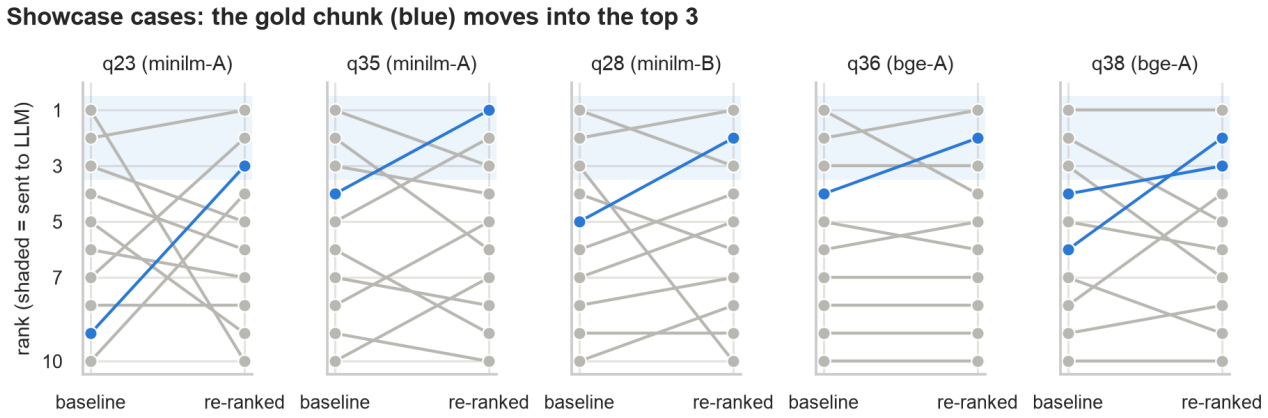


Figure 2: Rank of each top-10 chunk before and after re-ranking for the cases above. Gold in blue; shaded = top 3 sent to the LLM.

2.3 Where re-ranking failed

- **Regressions.** The cross-encoder can also be fooled by surface matches. For q22 ("umpire calls a wide, keeper whips the bails off") it ranked helmet and fielder clauses first, pushing the right one (22.9) from #2 to #5, and the answer went from correct to wrong. For q33 ("can the same bowler bowl the next Super Over?") a clause about the "same bowler" offending a third time (41.13.4) beat the Super Over rule (#3 → #7).
- **Lost chunks.** For 5 questions the gold chunk was below rank 10 in every setup, so the re-ranker never saw it. E.g. for "Is there a follow-on in T20?" the chunk just says "Shall not apply." and shares no words with the question.
- **Reading failures.** For q24, q31 and q40 the gold chunk was #1 and the answer was still wrong: the LLM read "However, … they shall refuse" as the rule (q24), "complete or replay" as "replay" (q40), and ignored "no more than 11 may bat" (q31). Retrieval can't fix that.

2.4 Is it just the LLM's own knowledge?

No. Without the document the LLM is **wrong 55% of the time** and almost never says it doesn't know. Splitting the questions into general cricket law vs. tournament-specific rules makes it clear (Figure 3): the model knows general laws (0.53) but not this tournament's rules (0.20). On those, RAG takes it to 0.64, and re-ranking to **0.73**. On general laws re-ranking slightly hurt (0.67 → 0.63), mostly because of the q22 regression.

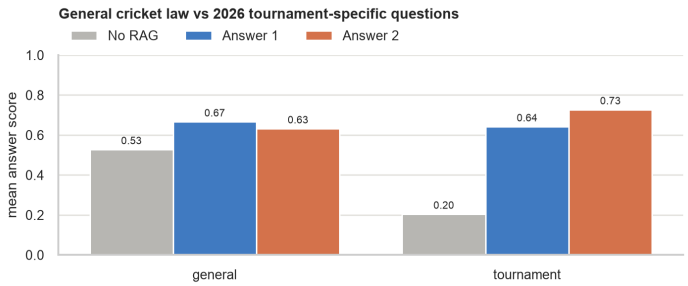


Figure 3: Mean answer score by question type and mode.

3 Latency Profiling

All on one RTX 2080 Ti (MiniLM + `ms-marco`), mean over 40 queries after 5 warm-ups, with `cuda.synchronize()` around each call (Table 4).

Re-ranking adds about **14 ms per query (3.2×)**, almost all of it the cross-encoder; the FAISS search itself is basically free. In absolute terms that's cheap, and small next to the LLM call (seconds), so for these accuracy gains it's clearly worth it. Batching queries brings Pipeline 2 down to 7 ms each.

stage	ms / query
query embedding	6.0
FAISS search (top 10)	0.05
cross-encoder on 10 pairs	14.2
Pipeline 1 (baseline retrieval)	**6.2**
Pipeline 2 (retrieval + re-ranking)	**19.8**
cross-encoder on all 440 chunks	295
Pipeline 2, 40 queries batched	7.3

Table 4: Retrieval latency per query on GPU.

4 Scaling: why not cross-encode everything?

Because the cost grows with the number of chunks, and nothing can be precomputed. A cross-encoder needs the question, so for every new query it has to run one forward pass per chunk. Here, scoring all 440 chunks took **295 ms per query**, about 6,000× slower than the FAISS search (0.05 ms). That's ∼0.67 ms per chunk, so a collection of 1M chunks would take about **11 minutes per query** on this GPU. Bi-encoder chunk embeddings are computed once, offline, and vector search (especially approximate indexes) stays in milliseconds even for millions of chunks.

So the two-stage setup gets most of the benefit for a fraction of the cost: the cheap bi-encoder narrows millions of chunks to 10, and the expensive cross-encoder only has to sort those 10. The trade-off, which my lost cases show, is that re-ranking can only fix what the first stage retrieves. A chunk that the bi-encoder ranks 100th stays lost.